

Análise do desempenho de uma aplicação de multiplicação de matrizes entre P-Core e E-Cores

JOÃO LUIZ GRAVE GROSS, Instituto de Informática da UFRGS, Brasil

[TODO]

Palavras-Chave: P-Core, E-Core, Intel

Formato de Referência da ACM:

João Luiz Grave Gross. 2025. Análise do desempenho de uma aplicação de multiplicação de matrizes entre P-Core e E-Cores. Disciplina: Computer System Performance Analysis (CMP223). (Novembro 2025), 4 páginas.

1 Introdução

[TODO] P-cores performance E-cores energia/econômico aplicação de multiplicação de matrizes

2 Plataforma: Hardware e SO

A plataforma de execução consiste em uma estação de trabalho Dell OptiPlex 7020 Micro. As especificações de *hardware* incluem uma CPU Intel Core i7-14700T (14ª Geração, 20 cores, sendo 8 *P-cores*/16 *threads* e 12 *E-cores*/12 *threads*, cache de 33 MB, 1.3GHZ a 5.0GHz), 16GB de RAM DDR5 5600MT/s (módulo único) e SSD de 512GB NVMe M.2 [1]. O ambiente de *software* baseia-se no sistema operacional Ubuntu 24.04 LTS [4].

O mini desktop em questão foi escolhido, pois o autor tem acesso irrestrito ao equipamento (24/7) e é usuário único, permitindo que os experimentos pudessem ser executados a qualquer tempo, sem a necessidade de compartilhamento de recursos. Adicionalmente, o equipamento está fisicamente instalado em um Data Center, onde a temperatura é mantida estável entre 17°C e 21°C, mitigando oscilações térmicas que poderiam interferir nos resultados.

3 Algoritmo de Multiplicação de Matrizes

A multiplicação de matrizes foi selecionada como a carga de trabalho base para a avaliação de desempenho entre *p-cores* e *e-cores*. Esta escolha fundamenta-se na natureza estritamente *CPU-bound* do problema e em sua alta capacidade de paralelização, características que permitem mensurar o desempenho computacional do processador e distribuir um conjunto de tarefas entre os múltiplos núcleos.

O algoritmo recebe como parâmetro de entrada a dimensão N e procede para a criação de duas matrizes $N \times N$ inicializadas com dados pseudoaleatórios. Ele então segmenta o problema, criando 8 tarefas distintas, onde cada uma é responsável por computar uma fatia equivalente do cálculo total. Ao término da execução paralela, o sistema realiza a concatenação dos 8 blocos de resultados parciais para compor a matriz final da multiplicação.

4 Características das Implementações

Foram desenvolvidas três implementações para o algoritmo de multiplicação de matrizes. Embora todas se fundamentem no alto nível de paralelismo inerente ao problema, cada uma adota uma estratégia de gerenciamento de memória distinta.

A primeira implementação foi executada sobre o interpretador *Python 3.12.12* com o módulo *multiprocessing*. Nesta abordagem, instanciam-se 8 processos filhos através da *system call* `fork()`.

Informações de Contato do Autor: João Luiz Grave Gross, jlkgross@inf.ufrgs.br, Instituto de Informática da UFRGS, Porto Alegre, Rio grande do Sul, Brasil.

O método beneficia-se do mecanismo de *Copy-on-Write* (CoW), onde o espaço de endereçamento do processo pai é compartilhado virtualmente com os filhos. Visto que as operações nas matrizes são estritamente de leitura, elimina-se a sobrecarga (*overhead*) de duplicação ou transferência de dados, otimizando o consumo de recursos.

A abordagem com Dask, também para o *Python 3.12.12*, baseia-se no particionamento das matrizes em blocos. Dadas as limitações de configuração explícita de afinidade por *worker* no comando nativo da *.matmul*, a estratégia de controle de CPU é aplicada no nível do processo pai. Utiliza-se o conceito de **Herança de Afinidade**: define-se a afinidade da CPU no processo principal antes da execução do *fork* que cria os *workers*. Por consequência, quando os *workers* são instanciados, o *Kernel* garante que eles herdem as restrições do pai, assegurando que as tarefas distribuídas pelo Dask sejam processadas estritamente pelos núcleos designados.

Por fim, a implementação com *threading*, desenvolvida para o *Python 3.14.0*, explora a remoção do *Global Interpreter Lock* (GIL), recurso nativo desta versão (modo *free-threading*) que viabiliza o paralelismo real entre *threads*. A memória do processo pai é compartilhada entre todas as *threads* através do mecanismo *Zero Copy*, ou seja, diferentemente do CoW, não há duplicação de páginas de memória mesmo em caso de escrita, pois o acesso é direto ao espaço global compartilhado.

Todas as implementações registram, ao término de cada execução, o horário e a temperatura da CPU. Ainda, a fim de assegurar a estabilidade dos resultados, o *governor* do processador foi configurado estritamente no modo *performance*, forçando a operação dos núcleos na frequência máxima de *clock* suportada.

5 Cenários dos Testes

A fim de medir a performance dos *p-cores* e *e-cores* ao execução a multiplicação de matrizes 4 cenários de testes foram projetados. Conforme mencionado na seção 3, 8 tarefas são criadas e escalonadas para computação, logo cada um dos cenários é composto por um conjunto de 8 núcleos de CPU:

Com o objetivo de avaliar o desempenho dos *P-cores* e *E-cores* na execução da multiplicação de matrizes, foram projetados quatro cenários de teste. Conforme detalhado na Seção 3, são geradas oito tarefas computacionais simultâneas, portanto, cada cenário é composto por um conjunto distinto de oito núcleos de processamento.

O *hardware* utilizado possui CPU com 20 núcleos e seus *IDs* lógicos variam de 0 a 27. *IDs* de 0 a 15 são de núcleos *P-core*, de modo que cada par sequencial de *IDs* (0-1, 2-3, ..., 14-15) compartilha os recursos de um mesmo núcleo físico. Já *IDs* de 16 a 27 correspondem aos núcleos *E-core*, onde cada identificador representa estritamente um núcleo físico.

Abaixo a descrição dos cenários de testes:

8P_HT_OFF: Utiliza 8 *P-cores* com *Hyper-Threading* desabilitado (1 *thread* por núcleo físico). Os *IDs* utilizados são {0, 2, 4, 6, 8, 10, 12, 14}.

4P_HT_ON: Utiliza 4 *P-cores* com *Hyper-Threading* habilitado (2 *threads* por núcleo físico), totalizando 8 *threads* lógicas. Os *IDs* utilizados são {0, 1, 2, 3, 4, 5, 6, 7}.

4P_4E_HT_OFF: Configuração híbrida com 4 *P-cores* e 4 *E-cores*. Os *IDs* utilizados combinam núcleos de performance {0, 2, 4, 6} e de eficiência {16, 17, 18, 19}.

8E: Utiliza 8 *E-cores* (1 *thread* por núcleo físico). Os *IDs* lógicos utilizados são {16, 17, 18, 19, 20, 21, 22, 23}.

Em todos os cenários avaliados, optou-se pela manutenção da tecnologia *Intel Turbo Boost* habilitada, permitindo que a frequência de operação atinga os valores máximos permitidos. A desabilitação deste recurso confinaria os núcleos à sua frequência base [3], o que comprometeria os experimentos. Embora reconheça-se que a elevação da temperatura da CPU possa induzir a redução

Table 1. Frequency of Special Characters

Non-English or Math	Frequency	Comments
Ø	1 in 1,000	For Swedish names
π	1 in 5	Common in math
\$	4 in 5	Used in business
Ψ_1^2	1 in 40,000	Unexplained usage

forçada da frequência (*thermal throttling*), este risco foi mitigado com intervalos de descanso entre as execuções consecutivas.

6 Configurações do Ambiente

[TODO] - pyenv e versões de python - variáveis de ambiente, BLAS, numpy uma thread - Comandos para execução do Jupyter notebook com variáveis de ambiente, seja python 3.12 ou Python 3.14

7 Projeto Experimental

- 3 scripts - 4 cenários - 5 tamanhos de matriz: máximo de 8000 para evitar problema de memória no script Dask - 30 repetições: para aproximação de uma distribuição gaussiana real e para aumentar intervalo de confiança
 - adição de cargas de warmup (descartadas na análise)
 - embaralhamento dos 600 experimentos + 8 cargas de warmup iniciais matrix_size 8000

8 Análise dos Resultados

8.0.1 Script 1.

9 Reprodutibilidade

- github: link - pilha de software: GUIX/NIX

10 Conclusões

11 Sectioning Commands

11.1 Subsection

This is a subsection.

11.1.1 Subsubsection. This is a subsubsection.

Paragraph. This is a paragraph.
Subparagraph This is a subparagraph.

12 Tables

The “acmart” document class includes the “booktabs” package — <https://ctan.org/pkg/booktabs> — for preparing high-quality tables.

Immediately following this sentence is the point at which Table 1 is included in the input file; compare the placement of the table here with the table in the printed output of this document.

To set a wider table, which takes up the whole width of the page’s live area, use the environment **table*** to enclose the table’s contents and the table caption. As with a single-column table, this wide table will “float” to a location deemed more desirable. Immediately following this sentence

Table 2. Some Typical Commands

Command	A Number	Comments
<code>\author</code>	100	Author
<code>\table</code>	300	For tables
<code>\table*</code>	400	For wider tables

is the point at which Table 2 is included in the input file; again, it is instructive to compare the placement of the table here with the table in the printed output of this document.

Always use `midrule` to separate table header rows from data rows, and use it only for this purpose. This enables assistive technologies to recognise table headers and support their users in navigating tables more easily.

13 Figures

The “figure” environment should be used for figures. One or more images can be placed within a figure. If your figure contains third-party material, you must clearly identify it as such, as shown in the example below.

Fig. 1. 1907 Franklin Model D roadster. Photograph by Harris & Ewing, Inc. [Public domain], via Wikimedia Commons. (<https://goo.gl/VLCRBB>).

Some examples. A paginated journal article [2].

References

[1] Dell. 2025. Desktop OptiPlex Micro. <https://www.dell.com/pt-br/shop/cty/pdp/spd/optiplex-7020-micro>.
[2] Michael Downes and Barbara Beeton. 2004. *The amsart, amsproc, and amsbook document classes*. American Mathematical Society. <http://www.ctan.org/pkg/amslatex>.
[3] Intel. 2025. How Intel Technologies Boost Your CPU’s Performance. <https://www.intel.com/content/www/us/en/gaming/resources/how-intel-technologies-boost-cpu-performance.html>.
[4] Ubuntu. 2025. ubuntu 24.03 LTS. <https://ubuntu.com/download/desktop>.